

I. Análise de Sistemas: Definição, Papel e Ciclo de Vida

A **Análise de Sistemas** é a disciplina que serve como ponto de partida para qualquer projeto de software. Ela atua como a ponte de comunicação que une o mundo dos negócios com a tecnologia. O objetivo principal da análise e projeto de sistemas é criar soluções tecnológicas que aumentem a eficiência e a eficácia dos processos de negócios de uma organização.

1. O Papel do Analista de Sistemas

O analista de sistemas é visto como um "arquiteto de software" que faz a "tradução" das necessidades do cliente em algo funcional. Suas principais responsabilidades incluem:

- **Levantar Requisitos:** Conversar com os usuários para entender o que o sistema precisa fazer.
- **Documentar:** Registrar todas as informações e regras de negócio de forma clara.
- **Modelar:** Criar diagramas e desenhos para visualizar o funcionamento do sistema (como um arquiteto faz um projeto de casa), utilizando ferramentas como o Diagrama de Entidade-Relacionamento (DER).
- **Comunicar:** Servir como ponte entre a equipe de desenvolvimento e o cliente para garantir o alinhamento.
- **Criar Especificações de Software:** Detalhar os requisitos funcionais e técnicos para os programadores.
- **Testar e Treinar:** Garantir que o software atenda às especificações e treinar os usuários finais sobre como utilizá-lo.

2. O Processo e o Ciclo de Vida

A análise de sistemas é um processo iterativo, com etapas que podem ser repetidas e ajustadas ao longo do tempo. O **Ciclo de Vida de um Sistema** inclui as seguintes fases:

1. **Análise:** Entendimento do problema.
2. **Projeto:** Desenho da solução (banco de dados, telas, etc.).
3. **Desenvolvimento:** Codificação e implementação.
4. **Implantação:** Instalação para o usuário final.
5. **Manutenção:** Correção de *bugs* e adição de novas funcionalidades.

3. Conceitos Fundamentais

- **Sistema de Informação (SI):** Conjunto de regras e procedimentos organizados para fornecer informações aos usuários. É composto por três etapas: **entrada de dados**, **processamento** e **saída da informação**, com um ciclo de **feedback** que pode influenciar novas entradas.
- **Dado vs. Informação:** **Dado** é o elemento bruto, a matéria-prima (ex: nome, número). **Informação** é o dado que passou por um processo de tratamento ou processamento, gerando significado e sendo útil.
- **Teoria Geral dos Sistemas (TGS):** Busca entender sistemas complexos, tratando-os como um conjunto de partes inter-relacionadas que trabalham juntas para atingir um objetivo comum. Um sistema é composto por **subsistemas** (partes menores que interagem).

II. Levantamento e Tipos de Requisitos (Aula 2)

Um **requisito** é a lista de desejos do cliente, uma descrição detalhada do que o sistema precisa ser ou fazer, devendo ser **claro, completo, consistente e não ambíguo**.

1. Tipos de Requisitos

- **Requisitos Funcionais:** Descrevem o que o sistema deve **FAZER** (as funcionalidades). Exemplos incluem permitir o cadastro de usuários, adicionar produtos ao carrinho, ou processar pagamentos.
- **Requisitos Não-Funcionais:** Descrevem **COMO** o sistema deve se comportar, focando em características. Exemplos incluem desempenho (tempo de resposta não exceder 2 segundos), segurança (criptografia, autenticação, proteção contra injeção de SQL) e usabilidade (acessibilidade em smartphones, facilidade de navegação).

2. Técnicas de Levantamento

O analista utiliza diversas técnicas para entender as necessidades do cliente:

- **Entrevista:** Conversa direta com os *stakeholders* (partes interessadas), sendo crucial preparar perguntas e ouvir atentamente.
- **Observação:** O analista observa o processo de trabalho atual para identificar problemas e oportunidades.
- **Análise de Documentos:** Estudo de manuais, relatórios e formulários existentes para entender regras de negócio e dados.
- **Questionários:** Usados para coletar informações de um grande número de pessoas.
- **Prototipagem:** Criação de protótipos para que os usuários visualizem o funcionamento do software e forneçam feedback.
- **Uso de Diagramas:** Como casos de uso ou de fluxo, que auxiliam na descrição das interações.

III. Bancos de Dados e Modelagem de Dados (Aulas 3, 4, 5, 6)

1. Conceitos de Bancos de Dados (BD)

Um **Banco de Dados** é um sistema organizado para armazenar, gerenciar e recuperar informações de forma segura e fácil de encontrar. O **Sistema Gerenciador de Banco de Dados (SGBD)**, como o **MySQL** (que é gratuito, popular e usado no curso), é o software que permite criar e gerenciar BDs, controlando o acesso e garantindo a segurança e organização dos dados.

Conceitos Fundamentais (Modelo Relacional):

- **Tabela:** Estrutura principal, organiza dados em linhas e colunas.
- **Coluna (Campo):** Tipo de informação armazenada (ex: preço, nome_do_produto).
- **Registro (Linha):** Conjunto completo de informações sobre um único item.
- **Chave Primária (Primary Key - PK):** Coluna especial que identifica unicamente cada registro.

O SGBD utiliza três **Níveis de Abstração de Dados** para que o usuário não se preocupe com a forma de armazenamento:

1. **Visão do Usuário:** Partes do BD que cada usuário pode acessar.
2. **Nível Conceitual:** Define quais dados estão armazenados e como se relacionam.
3. **Nível Físico:** Determina como os dados estão efetivamente armazenados.

2. Modelagem de Dados

A **Modelagem de Dados** é a etapa de desenho da estrutura do BD, visualizando como as informações se organizam e se conectam.

Diagrama de Entidade-Relacionamento (DER)

O DER é a ferramenta mais comum, utilizando três elementos principais:

- **Entidade:** Objeto, pessoa ou conceito sobre o qual se deseja armazenar dados, representado por um retângulo.
- **Atributo:** Característica da entidade, representada por um oval. A Chave Primária é sublinhada.
- **Relacionamento:** Conexão entre duas ou mais entidades, representada por um losango.

A **Cardinalidade** define quantos registros de uma entidade podem se relacionar com outra:

- **Um para Um (1:1):** Cada registro se relaciona com apenas um registro da outra entidade (Ex: Cliente e CPF).
- **Um para Muitos (1:N):** Um registro se relaciona com muitos registros da outra entidade (Ex: Um Cliente pode fazer muitos Pedidos).
- **Muitos para Muitos (N:N):** Muitos registros de uma entidade se relacionam com muitos da outra (Ex: Muitos Alunos podem fazer muitas Disciplinas).

3. Linguagem SQL

A SQL (Structured Query Language) é usada para manipular SGBDs relacionais.

DDL (Data Definition Language) – Linguagem de Definição de Dados

Usada para criar e gerenciar a **estrutura** do banco de dados.

- **CREATE:** Usado para criar tabelas, vistas e índices. Ex: `CREATE TABLE clientes (...)`.
- **ALTER:** Usado para modificar a estrutura (e.g., adicionar uma coluna).
- **DROP:** Usado para apagar tabelas ou outros objetos.

DML (Data Manipulation Language) – Linguagem de Manipulação de Dados

Usada para **interagir** com os dados. Corresponde às quatro operações básicas de persistência de dados, conhecidas como **CRUD**:

- **Create (INSERT):** Adicionar novos registros (linhas).
- **Read (SELECT):** Consultar dados existentes.
- **Update (UPDATE):** Modificar dados em registros já existentes (deve-se usar `WHERE` para evitar atualizar todos os registros).
- **Delete (DELETE):** Remover registros de uma tabela (deve-se usar `WHERE` para evitar apagar todos os registros).

IV. Ferramentas, Programação e Implementação (Aulas 1, 7)

O desenvolvimento prático no curso utiliza ferramentas padrão da indústria e gratuitas:

- **Visual Studio Code (VS Code):** Editor de código leve e poderoso.
- **Python:** Linguagem de programação de alto nível, com sintaxe simples e alta versatilidade em áreas como inteligência artificial e desenvolvimento web.

- **GitHub/GitHub Desktop:** Plataforma online para hospedar projetos de código, permitindo colaboração e controle de versão.
- **MySQL Workbench:** Aplicativo visual que facilita a conexão com o MySQL, criação de schemas e execução de scripts SQL.

Fundamentos de Programação em Python

Em Python, o código é estruturado usando variáveis, tipos de dados e estruturas de controle:

- **Tipos de Dados Comuns:** `str` (texto), `int` (números inteiros), `float` (números decimais) e `bool` (valores lógicos Verdadeiro/Falso).
- **Estruturas de Controle:** Permitem tomar decisões (`if`, `elif`, `else`) e repetir tarefas (`for`, `while`).
- **Funções:** Blocos de código que realizam tarefas específicas, ajudando a organizar o código e evitar repetições.

V. Desafios, Viabilidade e Modelagem Complementar

1. Viabilidade e Custo-Benefício

A **Viabilidade** é a avaliação da possibilidade de implementar o sistema proposto, considerando recursos (hardware/pessoal), prazos e tecnologias (viabilidade técnica, financeira e operacional).

A **Relação Custo-Benefício** avalia o valor que o sistema entregará em comparação com o investimento necessário. Esta análise é medida pelo **ROI (Return On Investment)**. $ROI = \frac{\text{Benefícios} - \text{Custos}}{\text{Custos}}$ Um ROI positivo indica que o investimento gerou um retorno maior do que o valor investido.

2. Comunicação e Conhecimento

A comunicação eficaz entre o analista e o usuário é um desafio, devido à falta de entendimento mútuo (o analista tem conhecimento técnico; o usuário, conhecimento de negócio).

- **Conhecimento Empírico:** Adquirido através da experiência prática e observação direta. O usuário tem conhecimento empírico dos processos de negócio.
- **Conhecimento Tácito:** Implícito nas habilidades e intuições, sendo difícil de formalizar. O analista pode ter conhecimento tácito sobre o design ideal do sistema.

A colaboração é fundamental para que o conhecimento seja compartilhado, e o envolvimento do usuário desde o início é crucial para manter a comunicação clara e aberta.

3. Modelagem Lógica Complementar (UML)

Além do DER, a análise de sistemas utiliza ferramentas **UML** (Unified Modeling Language) para modelar e documentar o design de sistemas:

- **Diagrama de Fluxo de Dados (DFD):** Modela a funcionalidade, representando entradas e saídas e como as informações fluem através dos processos.
- **Diagrama de Casos de Uso:** Descreve a interação entre os usuários (*atores*) e as funcionalidades que o sistema oferece.
- **Diagrama de Classes:** Modela a estrutura de sistemas orientados a objetos, descrevendo classes, seus atributos, métodos e as relações entre elas.

- **Diagrama de Sequência:** Modela o comportamento do sistema ao longo do tempo, mostrando como os objetos interagem e as mensagens que são trocadas.

A modelagem lógica é uma etapa fundamental que auxilia na compreensão dos objetivos do sistema e facilita a comunicação entre o analista e os programadores. As etapas práticas da modelagem culminam na **Implementação do sistema**, que envolve a programação do software, a configuração de hardware e a instalação do banco de dados, garantindo que o sistema atenda aos requisitos definidos.